Before moving on, let’s also recap what are the PROs and CONs of using tries, and when we should prefer a trie over a BST.

On the PROs side, compared to using BSTs or hash tables:

- The search time only depends on the length of the searched string.
- Search misses only involve examining a few characters (in particular just the longest common prefix between the search string and the corpus stored in the tree).
- There are no collisions of unique keys in a trie.
- There is no need to provide a hash function or to change hash functions as more keys are added to a trie.
- A trie can provide an alphabetical ordering of the entries by key.

As appalling as this list looks like, as we have repeated this many times unfortunately there is no perfect data structure. So even tries do have some downsides:

- Tries can be slower than hash tables for looking up data whenever our container is too big to fit in memory. Hash tables would need less disk accesses, even down to a single access, while a trie would require `O(m)` disk reads for a string of length `m`.
- Hash tables are usually allocated in a single big and contiguous chunk of memory, while tries nodes can span the whole heap. So, the former would better exploit the principle of locality.
- Tries ideal use case is storing text strings. We could, in theory, stringify any value, from numbers to objects, and store it. Yet, if we were to store floating point